
Modelo de Programação Linear

Planejamento Macroscópico — Ball Corporation

1. Minimização de custos • 2. Conjuntos • 3. Parâmetros • 4. Função Auxiliar
• 5. Variáveis • 6. Função Objetivo • 7. Restrições • 8. Formulacção Compacta
• 9. Implementação • 10. Análise de Sensibilidade
-

Modelo de minimização de custos

O modelo busca **minimizar o custo total de operação** da malha industrial da Ball Corporation, contemplando quatro componentes: produção, estocagem, transferência logística e penalidade por demanda não atendida. O modelo é estritamente contínuo (sem variáveis binárias ou inteiras) e cobre um horizonte de 15 dias.

$$\begin{aligned} \min Z = & \underbrace{\sum_{p \in P} \sum_{l \in L_p} \sum_{i \in I} \sum_{t \in T} k_{i,p,l} \cdot x_{i,p,l,t}}_{\text{custo de produção}} + \underbrace{\sum_{p \in P} \sum_{l \in L_p} \sum_{i \in I} \sum_{t \in T} v_{i,p,l} \cdot e_{i,p,l,t}}_{\text{custo de estoque}} \\ & + \underbrace{\sum_{i \in I} \sum_{o \in P} \sum_{\substack{d \in P \\ d \neq o}} \sum_{t \in T} ct_{o,d} \cdot \frac{f_{i,o,d,t}}{200.000}}_{\text{custo logístico}} + \underbrace{\sum_{p \in P} \sum_{l \in L_p} \sum_{i \in I} \sum_{t \in T} M \cdot w_{i,p,l,t}}_{\text{penalidade por venda perdida}} \end{aligned}$$

Sujeito às restrições R1–R9 detalhadas na Seção 7.

Conjuntos (Índices)

Símbolo	Descrição	Índice	Cardinalidade
I	SKUs (produtos finais)	i	30
P	Plantas (fábricas)	p, o, d	10
L_p	Linhas da planta p	l	3 por planta
L	Todas as linhas ($L = \bigcup_{p \in P} L_p$)	l	30
T	Períodos ($t_1, \dots, t_{15}$)	t	15 dias
F	Formatos de lata do portfólio	—	3
C	Triplas capáveis (i, p, l) : formato de $i = \text{for-}$ mato de $l \in L_p$	(i, p, l)	$\subseteq I \times L$

Conjunto $\mathcal{C}$ — Capabilidade. Os três formatos são: *9.1 Oz Sleek*, *12 Oz Standard* e *16 Oz Tall*. Cada planta p opera com exatamente 1 formato. Uma tripla (i, p, l) pertence a $\mathcal{C}$ se o formato do SKU i coincide com o formato da planta p . Fora de $\mathcal{C}$, a produção é proibida ($x_{i,p,l,t} = 0$ via R8). Os $\approx 10\%$ de pedidos incompatíveis são resolvidos via transferência $f_{i,o,d,t}$ de uma planta apta, seguida de redistribuição interna $g_{i,p,l,t}$.

Parâmetros

Logísticos (`input_logistic_costs_artificial.csv`)

$cb_{o,d}$, $cp_{o,d}$, $cl_{o,d}$: custos de combustível, pedágio e mão de obra da rota $o \rightarrow d$
 $ct_{o,d} = cb_{o,d} + cp_{o,d} + cl_{o,d}$ (custo total por caminhão) $ct_{o,o} = 0$

Produção (`input_production_rate` e `input_sku_costs`)

$r_{p,l,t}$: taxa de produção da linha $l \in L_p$ no dia t (latas/hora)

Indexado por linha e dia — não por SKU. Ausência de registro implica produção proibida.

$k_{i,p,l}$: custo de produção do SKU i na linha $l \in L_p$ (R\$/lata)

$v_{i,p,l}$: custo de estocagem do SKU i na linha $l \in L_p$ (R\$/lata/dia)

Demanda e armazenagem (`input_production_need` e `input_line_storage_limit`)

$\eta_{i,p,l,t}$: demanda do SKU i na linha $l \in L_p$ com prazo no dia t (latas)

Pode ser incompatível com o formato da planta (ruído de $\approx 10\%$).

$e_{p,l}^{\max}$: capacidade máxima de armazenagem da linha $l \in L_p$ (latas) — fixo.

$e_{i,p,l}^0$: estoque inicial do SKU i na linha $l \in L_p$. Neste estudo: $e_{i,p,l}^0 = 0 \forall i, p, l$.

Função Auxiliar

$$\phi : L \rightarrow P, \quad \phi(l) = p \quad (\text{linha } l \mapsto \text{planta } p)$$

Como $|L_p| = 3$ para todo p , a função ϕ é sobrejetora. Sempre que p e l aparecem juntos nos índices, vale $p = \phi(l)$.

Variáveis de Decisão

São sete variáveis de decisão, todas contínuas e não-negativas:

Variável	Descrição	Domínio
$x_{i,p,l,t}$	Volume produzido do SKU i na linha $l \in L_p$, no dia t . Só definida para $(i, p, l) \in \mathcal{C}$.	$\mathbb{R}_+$
$e_{i,p,l,t}$	Nível de estoque do SKU i na linha $l \in L_p$ ao final do dia t .	$\mathbb{R}_+$
$a_{i,p,l,t}$	Volume de atendimento local do SKU i a partir da linha $l \in L_p$ no dia t .	$\mathbb{R}_+$
$f_{i,o,d,t}$	Volume transferido do SKU i da planta o para a planta $d \neq o$, no dia t .	$\mathbb{R}_+$
$g_{i,p,l,t}$	Volume redistribuído internamente da doca de entrada da planta p para o armazém da linha l , no dia t .	$\mathbb{R}_+$
$s_{i,p,l,t}$	Volume retirado do armazém da linha $l \in L_p$ para a doca de saída da planta p , no dia t .	$\mathbb{R}_+$
$w_{i,p,l,t}$	Volume de demanda não atendida (venda perdida) do SKU i na linha $l \in L_p$, no dia t . Variável artificial de folga.	$\mathbb{R}_+$

Por que $f_{i,o,d,t}$ e não x com destino?

O frete é pago pelo **movimento físico entre plantas**, não pela produção. Se Jacareí produz 1.000.000 de latas e guarda tudo localmente por 3 dias, nenhum caminhão é usado — custo logístico zero. O caminho físico das latas é:

$$x_{i,p,l,t} \xrightarrow{\text{estoque}} e_{i,p,l,t} \xrightarrow{s: \text{doca de saída}} f_{i,p,d,t} \xrightarrow{g: \text{doca de entrada}} e_{i,d,l',t} \xrightarrow{a} \text{cliente}$$

Cada variável carrega seu próprio custo: $x \rightarrow k_{i,p,l}$, $e \rightarrow v_{i,p,l}$, $f \rightarrow ct_{o,d}$.

O nó de transbordo: por que g e s ?

Caminhões chegam e saem pelo pátio da fábrica (a doca da planta p), não diretamente na porta de cada linha. As variáveis g e s modelam as **empilhadeiras** que fazem a distribuição interna:

- $g_{i,p,l,t}$: caminhão descarrega no pátio $\rightarrow$ empilhadeira leva ao armazém da linha l .
- $s_{i,p,l,t}$: empilhadeira retira do armazém da linha $l \rightarrow$ leva à doca para encher caminhão.

Por que não usar f direto no balanço de estoque? Se sumássemos $\sum_o f_{i,o,p,t}$ direto na equação de cada linha l , o solver *clonaria* as latas recebidas para todos os armazéns simultaneamente. As restrições R6 e R7 garantem o **Princípio da Conservação de Massa** no transbordo: o que entra no pátio sai exatamente para os armazéns, e vice-versa.

Como o estoque $e_{i,p,l,t}$ se forma — balanço intuitivo

$$e_{i,p,l,t} = \underbrace{e_{i,p,l,t-1}}_{\text{estoque anterior}} + \underbrace{x_{i,p,l,t}}_{\text{produção}} + \underbrace{g_{i,p,l,t}}_{\text{entrada da doca}} - \underbrace{s_{i,p,l,t}}_{\text{saída para doca}} - \underbrace{a_{i,p,l,t}}_{\text{atendimento local}}$$

Formalizado como restrição de igualdade em R2, com condição inicial $e_{i,p,l,0} = e_{i,p,l}^0 = 0$.

O que é $w_{i,p,l,t}$ e por que ela existe? — A lógica do Big- M

No mundo real, quando a demanda excede a capacidade física, ocorre **venda perdida** — não o colapso do universo. Para evitar que o modelo declare *infeasible* em cenários de estresse (ex: 150% de ocupação), introduzimos w como uma **variável artificial de folga**.

O balanço econômico (Big- M): Se w custasse zero, o solver *deixaria de produzir* e jogaria toda a demanda em w (pois produzir custa dinheiro). Punimos w com um custo $M = R\$1.000.000$ por lata não entregue — tão alto que o modelo só recorre a w quando esgota todas as possibilidades físicas. Esse mecanismo é conhecido em Pesquisa Operacional como **penalidade Big- M** .

Guia das Variáveis: o que cada uma representa fisicamente

Esta seção é destinada a quem conhece Programação Linear mas não conhece a operação de uma malha de fábricas de latas. O objetivo é tornar claro **por que** cada variável existe e o que ela representa no chão de fábrica — antes de vê-la numa equação.

$a_{i,p,l,t}$ — Atendimento local: a única variável que toca o cliente

No modelo, **nenhuma outra variável entrega lata na mão do cliente**. x produz, e guarda, f transporta, g distribui internamente, s coleta para expedir — mas nenhuma delas encerra o pedido. Quem encerra é $a_{i,p,l,t}$.

Pense assim: um pedido chega na linha JAC_L2 exigindo 50.000 latas de Brahma_Lata até o dia 5. O pedido só é considerado atendido quando $a_{\text{Brahma,JAC,JAC_L2,5}} \geq 50.000$. O caminho que essas latas percorreram até chegar lá — produzidas localmente, chegadas de Extrema via transferência, ou acumuladas desde o dia 3 — não importa para o cliente. Importa que a seja suficiente.

Por que a é indexado por linha e não por planta? Porque a demanda $\eta_{i,p,l,t}$ chegou amarrada a uma linha específica (campo `production_line` no CSV). O atendimento precisa ser rastreado na mesma granularidade do pedido para que a restrição R3 ($a + w = \eta$) faça sentido físico.

O que a não é: a não é produção. Uma linha pode atender um pedido inteiramente com latas que vieram de outra planta via $f \rightarrow g$, sem produzir nada localmente ($x = 0$). Nesse caso, $a > 0$ mas $x = 0$ — algo impossível de modelar se confundirmos as duas variáveis.

$e_{i,p,l,t}$ — Por que o estoque é da linha, não da planta?

Quem conhece modelos de rede esperaria que o estoque fosse indexado pela planta p — afinal, o armazém é físico e pertence à fábrica. Mas aqui o estoque é indexado pela **linha** l , e há uma razão precisa para isso.

A demanda $\eta_{i,p,l,t}$ chega amarrada a uma linha específica. O atendimento $a_{i,p,l,t}$ também é por linha. Para que o balanço de estoque R2 feche corretamente ($e_t = e_{t-1} + x + g - s - a$), o estoque precisa estar na mesma granularidade de a e x — ou seja, por linha.

Analogia: imagine que cada linha é um corredor separado de supermercado. O cliente que pediu latas no corredor A não aceita receber do corredor B sem antes algum funcionário transferir a mercadoria entre corredores (isso é o g — a redistribuição interna).

$g_{i,p,l,t}$ e $s_{i,p,l,t}$ — As empilhadeiras que ninguém vê

Essas são as variáveis mais “escondidas” do modelo. Elas existem por uma necessidade matemática clara: f é indexado por **planta** ($o \rightarrow d$), mas o estoque é indexado por **linha** (l dentro de p). Precisa haver algo que faça a ponte entre os dois níveis hierárquicos.

Variável	O que representa	O que aconteceria sem ela
$g_{i,p,l,t}$	Latas descarregadas no pátio de p levadas ao armazém da linha l	O solver “clonaria” o frete recebido para todas as linhas ao mesmo tempo, multiplicando latas do nada
$s_{i,p,l,t}$	Latas retiradas do armazém da linha l levadas à doca de expedição de p	Não haveria como saber de qual linha saíram as latas expedidas, impossibilitando o balanço por linha

As restrições R6 e R7 são as “cancelas” que garantem conservação de massa: tudo que chega via f deve sair via g ; tudo que sai via f deve ter sido coletado via s . Sem essas cancelas, o modelo poderia *criar* ou *destruir* latas no transbordo.

$f_{i,o,d,t}$ — Transferência: quando e por que o modelo decide usá-la?

f existe para resolver dois problemas do mundo real:

1. **O ruído de formato ($\approx 10\%$ dos pedidos):** quando um pedido chega a uma linha que não sabe produzir aquele formato (ex: pedido de *9.1 Oz Sleek* para uma linha de *12 Oz Standard*), a única saída do modelo é buscar outra planta apta e transferir. O solver descobre isso sozinho: $x = 0$ por R8, então para atender $a \geq \eta$ via R3, precisa de $g > 0$, que exige $f > 0$ via R6.
2. **Arbitragem de capacidade:** mesmo sem ruído, pode ser ótimo produzir num lugar mais barato e transferir. Se a planta de Extrema tem custo de produção menor e folga produtiva, o modelo pode decidir produzir lá e mandar caminhões para Jacareí, desde que o custo do frete ($ct_{o,d}$) não anule a economia.

Por que f não é indexado por linha? O caminhão não sabe (nem precisa saber) para qual linha dentro da planta destino as latas vão. Ele descarrega no pátio e as empilhadeiras (g) decidem. Indexar f por linha forçaria a origem a conhecer a organização interna do destino — uma informação desnecessária e acoplamento errado entre plantas.

$w_{i,p,l,t}$ — Venda perdida: a variável que evita o *infeasible*

Em cenários onde a demanda excede a capacidade física total da malha, um modelo sem w simplesmente **não tem solução** — ele declara *infeasible* e para. Isso é um problema porque:

- A inviabilidade não diz *o quanto* a malha ficou devendo.
- Não é possível comparar cenários: um cenário inviável não tem custo ótimo para comparar com outro.
- Não é possível fazer análise de sensibilidade.

w transforma o problema inviável em **sempre viável**: qualquer demanda pode ser “atendida” por a ou “absorvida” por w . O truque é a penalidade Big- $M = R\$ 10^6$ por lata em w — tão cara que o solver só recorre a ela quando esgota todo o potencial produtivo e logístico da rede.

Como ler os resultados: se `custo_penalidade = 0` no output, a malha atendeu 100% da demanda. Se for positivo, $w > 0$ em alguma linha/dia — a razão é física (capacidade insuficiente), não matemática.

Função Objetivo

$$\begin{aligned} \min Z = & \sum_{p \in P} \sum_{\substack{l \in L_p \\ (i,p,l) \in \mathcal{C}}} \sum_{i \in I} \sum_{t \in T} k_{i,p,l} \cdot x_{i,p,l,t} + \sum_{p \in P} \sum_{l \in L_p} \sum_{i \in I} \sum_{t \in T} v_{i,p,l} \cdot e_{i,p,l,t} \\ & + \sum_{i \in I} \sum_{o \in P} \sum_{\substack{d \in P \\ d \neq o}} \sum_{t \in T} ct_{o,d} \cdot \frac{f_{i,o,d,t}}{200.000} + \sum_{p \in P} \sum_{l \in L_p} \sum_{i \in I} \sum_{t \in T} M \cdot w_{i,p,l,t} \end{aligned}$$

Termo	Fórmula	Interpretação
Produção	$k_{i,p,l} \cdot x_{i,p,l,t}$	Custo unitário (R\$/lata) $\times$ volume produzido
Estoque	$v_{i,p,l} \cdot e_{i,p,l,t}$	Custo de carregar 1 lata no armazém por 1 dia
Logística	$ct_{o,d} \cdot \frac{f_{i,o,d,t}}{200.000}$	Custo por caminhão $\times$ número de caminhões
Venda perdida	$M \cdot w_{i,p,l,t}$	Penalidade Big- M por demanda não atendida

Capacidade do caminhão: $25 \text{ pallets} \times 8.000 \text{ latas} = 200.000 \text{ latas/caminhão}$

Restrições

R1 — Limite de armazenagem.

O espaço físico do armazém é compartilhado entre todos os SKUs. A soma do estoque de todos os produtos não pode ultrapassar a capacidade da linha:

$$\sum_{i \in I} e_{i,p,l,t} \leq e_{p,l}^{\max} \quad \forall p \in P, l \in L_p, t \in T$$

R2 — Balanço de estoque.

Equação de conservação de massa do armazém da linha. Unificada usando $e_{i,p,l,0} = e_{i,p,l}^0 = 0$:

$$e_{i,p,l,t} = e_{i,p,l,t-1} + x_{i,p,l,t} + g_{i,p,l,t} - s_{i,p,l,t} - a_{i,p,l,t} \quad \forall i \in I, p \in P, l \in L_p, t \in T$$

Esta é uma igualdade estrita que **acopla os 15 períodos**: o estoque de hoje é a condição inicial de amanhã. O frete (f) não aparece aqui diretamente — ele entra via g (entrada) e sai via s (saída), garantindo que a redistribuição interna é tratada pelas docas (R6 e R7).

R3 — Atendimento à demanda.

O atendimento local a somado à venda perdida w deve igualar exatamente a demanda:

$$a_{i,p,l,t} + w_{i,p,l,t} = \eta_{i,p,l,t} \quad \forall i \in I, p \in P, l \in L_p, t \in T$$

O frete e a produção **não aparecem aqui** para evitar dupla contagem: eles já alimentaram o estoque via R2, e o estoque alimenta a . O solver só aciona w quando esgota os 24h produtivos de todas as linhas.

R4 — Capacidade produtiva (turno de 24 h).

A soma das horas consumidas por todos os SKUs na linha l não pode ultrapassar 24 h. O tempo para produzir $x_{i,p,l,t}$ latas é $x_{i,p,l,t}/r_{p,l,t}$ horas:

$$\sum_{i \in I} \frac{x_{i,p,l,t}}{r_{p,l,t}} \leq 24 \quad \forall p \in P, l \in L_p, t \in T$$

Para $(i, p, l) \notin \mathcal{C}$, temos $x_{i,p,l,t} = 0$ por R8, logo esses termos não contribuem para a soma.

R5 — Capacidade logística de expedição.

Cada planta pode expedir no máximo 80 caminhões por dia (saída), somando todos os SKUs e destinos:

$$\sum_{i \in I} \sum_{\substack{d \in P \\ d \neq p}} \frac{f_{i,p,d,t}}{200.000} \leq 80 \quad \forall p \in P, t \in T$$

R6 e R7 — Conservação de massa no transbordo (docas).

Tudo que os caminhões descarregam no pátio deve ser exatamente distribuído pelos armazéns das linhas (R6), e tudo que os armazéns enviam ao pátio deve ser exatamente carregado nos caminhões (R7):

$$\sum_{l \in L_p} g_{i,p,l,t} = \sum_{\substack{o \in P \\ o \neq p}} f_{i,o,p,t} \quad \forall i \in I, p \in P, t \in T \quad (\text{R6})$$

$$\sum_{l \in L_p} s_{i,p,l,t} = \sum_{\substack{d \in P \\ d \neq p}} f_{i,p,d,t} \quad \forall i \in I, p \in P, t \in T \quad (\text{R7})$$

R8 — Capabilidade de formato.

Linhas só podem produzir SKUs compatíveis com seu formato:

$$x_{i,p,l,t} = 0 \quad \forall (i,p,l) \notin \mathcal{C}, t \in T$$

R9 — Não-negatividade.

$$x_{i,p,l,t}, e_{i,p,l,t}, a_{i,p,l,t}, f_{i,o,d,t}, g_{i,p,l,t}, s_{i,p,l,t}, w_{i,p,l,t} \geq 0$$

Formulação Compacta do PL

min

$$Z = \sum_{p,l,i,t} k_{i,p,l} \cdot x_{i,p,l,t} + \sum_{p,l,i,t} v_{i,p,l} \cdot e_{i,p,l,t} + \sum_{i,o,d,t} ct_{o,d} \cdot \frac{f_{i,o,d,t}}{200.000} + \sum_{p,l,i,t} M \cdot w_{i,p,l,t}$$

s.a.

$$\sum_i e_{i,p,l,t} \leq e_{p,l}^{\max} \quad \forall p, l, t \quad (\text{R1})$$

$$e_{i,p,l,t} = e_{i,p,l,t-1} + x_{i,p,l,t} + g_{i,p,l,t} - s_{i,p,l,t} - a_{i,p,l,t} \quad \forall i, p, l, t \quad (\text{R2})$$

$$a_{i,p,l,t} + w_{i,p,l,t} = \eta_{i,p,l,t} \quad \forall i, p, l, t \quad (\text{R3})$$

$$\sum_i \frac{x_{i,p,l,t}}{r_{p,l,t}} \leq 24 \quad \forall p, l, t \quad (\text{R4})$$

$$\sum_{i,d \neq p} \frac{f_{i,p,d,t}}{200.000} \leq 80 \quad \forall p, t \quad (\text{R5})$$

$$\sum_{l \in L_p} g_{i,p,l,t} = \sum_{o \neq p} f_{i,o,p,t} \quad \forall i, p, t \quad (\text{R6})$$

$$\sum_{l \in L_p} s_{i,p,l,t} = \sum_{d \neq p} f_{i,p,d,t} \quad \forall i, p, t \quad (\text{R7})$$

$$x_{i,p,l,t} = 0 \quad \forall (i, p, l) \notin \mathcal{C}, t \quad (\text{R8})$$

$$x, e, a, f, g, s, w \geq 0 \quad \forall i, p, l, o, d, t \quad (\text{R9})$$

$6 \times 30 \times 10 \times 3 \times 15 = \mathbf{81.000}$ variáveis operacionais + $30 \times 100 \times 15 = \mathbf{45.000}$ variáveis de transferência $\approx \mathbf{126.000}$ variáveis no total.

Implementação em Julia/JuMP

Esta seção explica como o modelo matemático se traduz em código, passo a passo. O objetivo é que mesmo quem nunca programou em Julia consiga entender a lógica por trás de cada bloco.

9.1 Pacotes utilizados

```
using JuMP          # linguagem de modelagem matematica (escreve o LP)
using Gurobi         # solver comercial que resolve o LP
using CSV            # leitura de arquivos .csv
using DataFrames     # estrutura de tabela (como um Excel no código)
using Printf         # formatacao de numeros no terminal
using Dates          # data/hora para nomear os arquivos de saida
```

O que é JuMP? JuMP é uma linguagem de modelagem embutida no Julia. Em vez de implementar o simplex manualmente, você **descreve** o modelo (variáveis, restrições, objetivo) e o JuMP monta a matriz LP e entrega ao solver. O Gurobi é o solver — o motor matemático que de fato encontra a solução ótima.

9.2 Leitura e preparação dos dados

```
# Extrai o dia da string de data (ex: "2024-01-05" -> 5)
function extrair_dia(data_str::String)
    return parse{Int, string}(data_str)[9:10]
end

# Converte numero brasileiro "1.234,56" para Float64 1234.56
function parse_br_float(val)
    ismissing(val) || val == "" && return 0.0
    return parse{Float64, replace(String(val), ", " => ".")}(val)
end
```

Por que essas funções? Os CSVs usam o padrão brasileiro de números (vírgula como decimal). O Julia espera ponto. Sem a conversão, `parse` lançaria um erro. A extração do dia resolve o fato de que o campo `deadline` vem como data completa ("2024-01-05"), mas o modelo usa $t \in \{1, \dots, 15\}$.

9.3 Extração dos conjuntos

```

I    = unique(df_need.product_id)    # conjunto I: 30 SKUs
P    = unique(df_rate.plant)         # conjunto P: 10 plantas
T    = 1:15                          # conjunto T: dias 1 a 15

# L_p: dicionario planta -> lista de linhas daquela planta
L_p = Dict{p => unique(df_rate[df_rate.plant .== p,
                        :production_line]) for p in P}

# Conjunto C: triplas (i, p, l) onde o formato do SKU bate
# com o formato que a linha sabe produzir
formatos_da_linha = Dict{r.production_line => r.sku_size_shape
                        for r in eachrow(df_rate)}

C = Set(
    (r.product_id, r.plant, r.production_line)
    for r in eachrow(df_need)
    if haskey(formatos_da_linha, r.production_line) &&
        r.sku_size_shape == formatos_da_linha[r.production_line]
)

```

O Dict como L_p e o Set como C : O dicionário L_p implementa L_p diretamente — iterar `for p in P, l in L_p[p]` é exatamente $\sum_{p \in P} \sum_{l \in L_p}$. O Set C armazena as triplas capáveis e permite verificação $(i, p, l) \in C$ em tempo $O(1)$ por hash, crítico dado que ocorre em cada iteração dos somatórios.

9.4 Construção dos parâmetros

```

# parse_br_float aplicado em TODOS os campos numericos dos CSVs
k      = Dict((r.product_id, r.plant, r.production_line) =>
              parse_br_float(r.production_cost) for r in eachrow(
                  df_costs))
v      = Dict((r.product_id, r.plant, r.production_line) =>
              parse_br_float(r.inventory_cost)  for r in eachrow(
                  df_costs))
ct     = Dict((r.origem, r.destino) =>
              parse_br_float(r.custo_total_frete) for r in eachrow(
                  df_log))
e_max  = Dict((r.plant, r.production_line) =>
              parse_br_float(r.storage_limit)  for r in eachrow(
                  df_limit))

# Demanda: inicializa com 0 para cobrir combinacoes sem pedido no
# CSV
eta = Dict{Tuple{Any,Any,Any,Int}, Float64}()
for i in I, p in P, l in L_p[p], t in T
    eta[(i, p, l, t)] = 0.0
end
for r in eachrow(df_need)
    t = extrair_dia(string(r.deadline))
    chave = (r.product_id, r.plant, r.production_line, t)
    haskey(eta, chave) && (eta[chave] += r.prod_need)
end

```

Por que parse_br_float em todos os parâmetros? A versão anterior só convertia a coluna prod_need. Os campos production_cost, inventory_cost, custo_total_frete e storage_limit também podem usar vírgula decimal nos CSVs brasileiros. Sem a conversão, esses valores seriam lidos como String e causariam erros silenciosos ou crash na montagem do modelo.

9.5 Variáveis de decisão no JuMP

```

modelo = Model(Gurobi.Optimizer)
set_silent(modelo) # suprime o log do Gurobi

@variable(modelo, x[i in I, p in P, l in L_p[p], t in T] >= 0)
@variable(modelo, e[i in I, p in P, l in L_p[p], t in T] >= 0)
@variable(modelo, a[i in I, p in P, l in L_p[p], t in T] >= 0)
@variable(modelo, f[i in I, o in P, d in P, t in T; o != d] >= 0)
@variable(modelo, g[i in I, p in P, l in L_p[p], t in T] >= 0)
@variable(modelo, s[i in I, p in P, l in L_p[p], t in T] >= 0)
@variable(modelo, w[i in I, p in P, l in L_p[p], t in T] >= 0)

```

Como ler @variable: Cada @variable cria automaticamente uma variável por combinação de índices. Por exemplo, `x[i in I, p in P, l in L_p[p], t in T]` cria $30 \times 10 \times 3 \times 15 = 13.500$ variáveis. O `o != d` em `f` garante que transferência para a mesma planta não existe — implementando diretamente a condição $o \neq d$ do modelo. O `>= 0` implementa R9 automaticamente.

9.6 Função objetivo (com $\mathcal{C}$ explícito)

```
@objective(modelo, Min,
  # Producao: so triplas em C contribuem
  sum(k[(i,p,l)] * x[i,p,l,t]
    for p in P, l in L_p[p], i in I, t in T
    if (i,p,l) in C) +
  # Estoque: todos os armazens
  sum(v[(i,p,l)] * e[i,p,l,t]
    for p in P, l in L_p[p], i in I, t in T
    if haskey(v,(i,p,l))) +
  # Logistica: custo por caminhao
  sum(ct[(o,d)] * (f[i,o,d,t] / 200_000)
    for i in I, o in P, d in P, t in T if o != d) +
  # Big-M: penalidade por venda perdida
  sum(1_000_000 * w[i,p,l,t]
    for p in P, l in L_p[p], i in I, t in T)
)
```

Conjunto $\mathcal{C}$ vs haskey: A versão anterior usava `haskey(k,(i,p,l))` como proxy para $\mathcal{C}$. O código correto usa `(i,p,l) in C`, onde C foi construído explicitamente comparando formatos. Isso é mais correto: um par (i,p,l) pode existir no dicionário k sem ser compatível em formato. O Big- $M = 10^6$ garante que w só é acionado quando a fábrica esgota toda capacidade física.

9.7 Restrições

```

# R1: limite de armazenagem (espaco compartilhado entre SKUs)
@constraint(modelo, R1[p in P, l in L_p[p], t in T],
    sum(e[i,p,l,t] for i in I) <= e_max[(p,l)])

# R2: balanço de estoque (e[t=0] = 0 via operador ternario)
@constraint(modelo, R2[i in I, p in P, l in L_p[p], t in T],
    e[i,p,l,t] == (t == 1 ? 0.0 : e[i,p,l,t-1]) +
        x[i,p,l,t] + g[i,p,l,t] - s[i,p,l,t] - a[i,p,l,t]
    )

# R3: atendimento a demanda com fator de escalonamento
@constraint(modelo, R3[i in I, p in P, l in L_p[p], t in T],
    a[i,p,l,t] + w[i,p,l,t] == eta[(i,p,l,t)] * fator_demanda)

# R4: capacidade produtiva (planta quebrada => limite 0h)
@constraint(modelo, R4[p in P, l in L_p[p], t in T],
    sum(x[i,p,l,t] / r_taxa[(p,l,t)]
        for i in I if (i,p,l) in C && r_taxa[(p,l,t)] > 0) <=
    (p == fabrica_quebrada ? 0.0 : 24.0))

# R5: caminhões de saída por planta por dia
@constraint(modelo, R5[p in P, t in T],
    sum(f[i,p,d,t] / 200_000
        for i in I, d in P if d != p) <= limite_caminhoes)

# R6: doca de ENTRADA (frete chegando -> g redistribuindo pelas linhas)
@constraint(modelo, R6[i in I, p in P, t in T],
    sum(g[i,p,l,t] for l in L_p[p]) ==
    sum(f[i,o,p,t] for o in P if o != p))

# R7: doca de SAÍDA (s coletando das linhas -> frete saindo)
@constraint(modelo, R7[i in I, p in P, t in T],
    sum(s[i,p,l,t] for l in L_p[p]) ==
    sum(f[i,p,d,t] for d in P if d != p))

# R8: capacidade de formato (usa conjunto C, não haskey)
@constraint(modelo, R8[i in I, p in P, l in L_p[p], t in T;
    (i,p,l) not in C],
    x[i,p,l,t] == 0)

```

Notas sobre as restrições:

- **R2 com $t == 1$:** o operador ternário implementa $e^0 = 0$ sem separar em dois blocos de constraint.
- **R4 com (i, p, l) in C :** a versão anterior usava `haskey(k, (i, p, l))`. O código correto usa o conjunto C explícito baseado em formato.
- **R4 planta quebrada:** `p == fabrica_quebrada ? 0.0 : 24.0` zera o limite de horas da planta, desligando-a completamente.
- **R6 = entrada, R7 = saída:** os rótulos estão alinhados com o código Julia. R6 redistribui o frete recebido ($f \rightarrow g$); R7 coleta o frete a expedir ($s \rightarrow f$).
- **R8 com C :** $(i, p, l) \notin C$ filtra as variáveis que devem ser forçadas a zero por incompatibilidade de formato.

Painel de Controle e Análise de Sensibilidade

10.1 O que é Análise de Sensibilidade em Pesquisa Operacional?

Um modelo de otimização resolve um problema para um conjunto **fixo** de parâmetros. Mas no mundo real, os parâmetros mudam: a demanda cresce, equipamentos quebram, caminhões entram em greve. A **Análise de Sensibilidade** responde:

“Até quanto posso apertar um parâmetro antes de a solução ótima mudar drasticamente?”

Em vez de refazer o modelo do zero, variamos os parâmetros de controle e observamos como o custo ótimo, a taxa de atendimento e o uso logístico respondem. Essa é a base do **Painel de Controle** implementado no código.

10.2 Os três eixos de perturbação

Parâmetro	O que representa	Como perturbar
fator_demanda	Multiplica toda a demanda $\eta_{i,p,l,t}$ por um fator. 1.0 = normal, 2.0 = dobra a demanda.	Simula choques de mercado ou promoções. Combinado com <code>ocupacao_100</code> , equivale a testar ocupações além dos 150% dos dados.
fabrica_quebrada	Reduz o limite de horas da planta escolhida de 24 h para 0 h em R4.	Simula parada de manutenção, greve ou desastre. Revela quais plantas são críticas para a malha.
limite_caminhoes	Substitui o valor 80 na restrição R5.	Simula greve de transportadores ou escassez de veículos. Revela o quanto a rede depende do frete inter-planta.

10.3 Estratégia de experimentação sugerida

O script roda os 21 cenários (`ocupacao_50` a `ocupacao_150`) com a configuração fixa do painel. Para uma análise completa, sugere-se rodar em sequência:

Rodada	fator	fabrica	limite	Objetivo
1	1.0	Nenhuma	80	Baseline — comportamento normal da malha
2	2.0	Nenhuma	80	Choque de demanda — onde aparece w ?
3	1.0	Extrema	80	Planta crítica — impacto no custo?
4	1.0	Nenhuma	10	Colapso logístico — a rede suporta?
5	2.0	Extrema	10	Cenário catastrófico — até onde vai w ?

10.4 Lendo o output

Cada rodada exporta um CSV nomeado automaticamente com a configuração usada, contendo uma linha por cenário de ocupação. Colunas-chave:

- **custo_producao, custo_estoque, custo_logistica:** decomposição do custo real — identifica qual componente cresce mais com a ocupação.
- **custo_penalidade:** valor total pago em Big- M . Zero = 100% de atendimento. Positivo = venda perdida.
- **taxa_atendimento:** percentual da demanda efetivamente entregue (a/η).
- **caminhoes:** total de caminhões despachados. Se atingir $80 \times 10 \times 15 = 12.000$, R5 está saturada.

O código completo está no arquivo `analise_cenarios.jl`. Ajuste apenas `caminho_base` e as três variáveis do Painel de Controle antes de rodar.